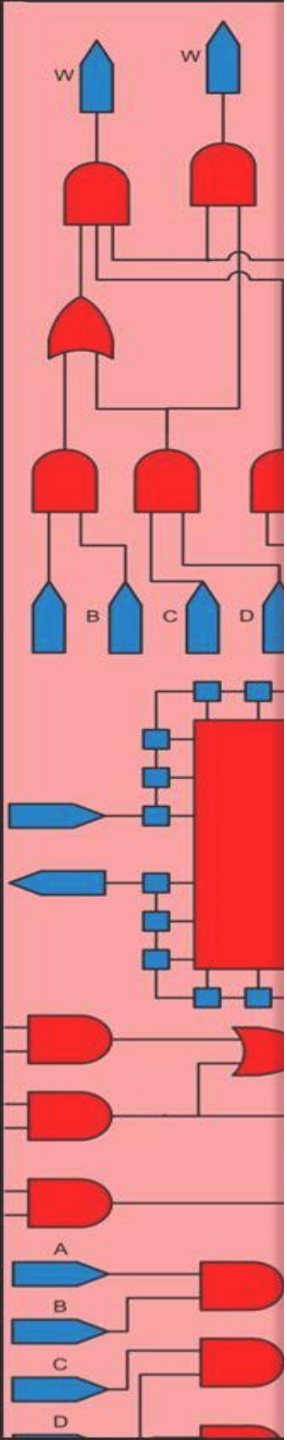


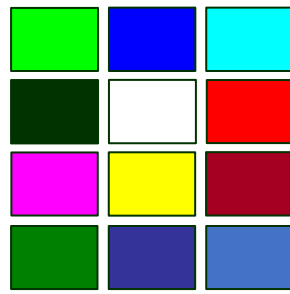
Topic 2

Combinational Components



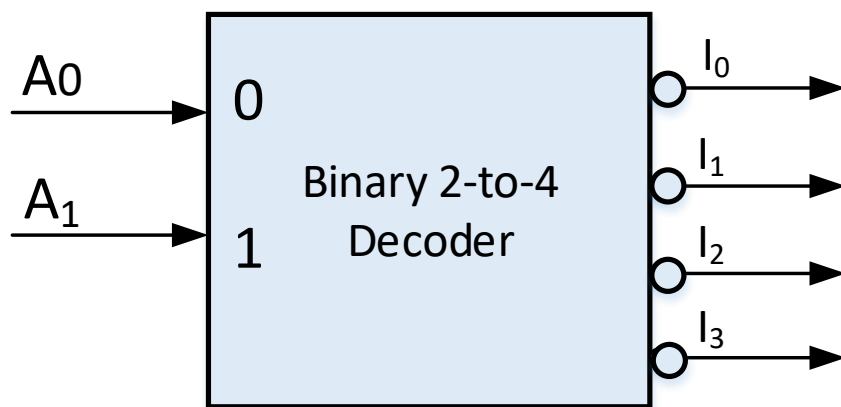
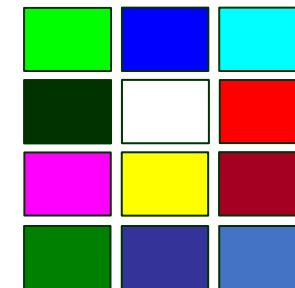
Combinational Components

- Decoder
- Multiplexer
- Comparator
- Adder
- Shifter
- ALU
- Case Study
 - Max Finder
 - One's Counter



Combinational components

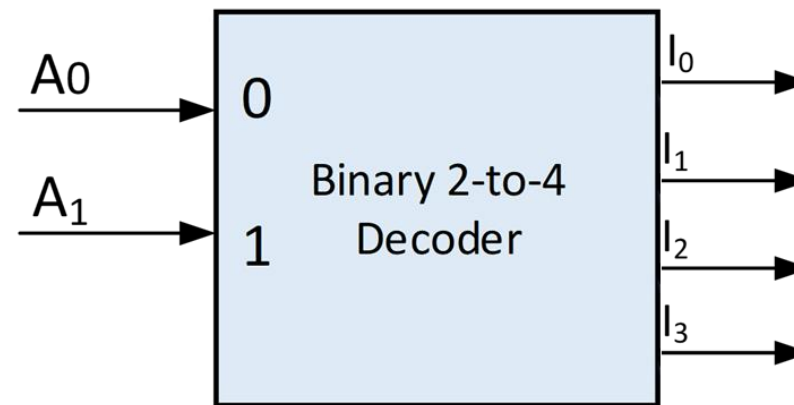
Decoder (functionality and gate level)



A_0	A_1	I_0	I_1	I_2	I_3
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

One-cold Encoding

Active low outputs



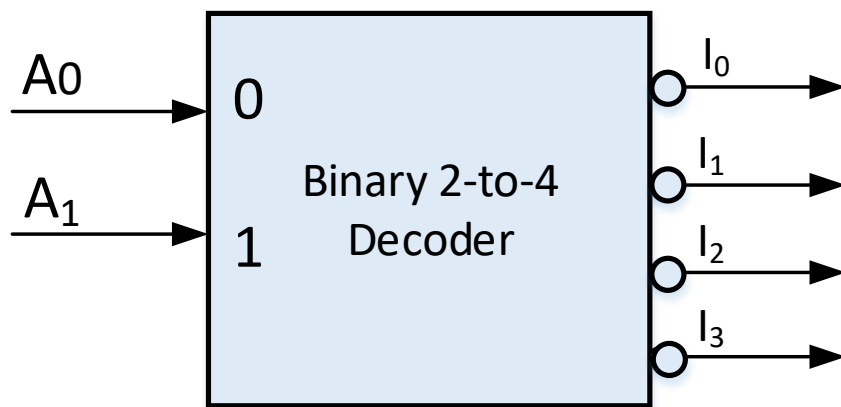
A_0	A_1	I_0	I_1	I_2	I_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

One-hot Encoding

Active high outputs

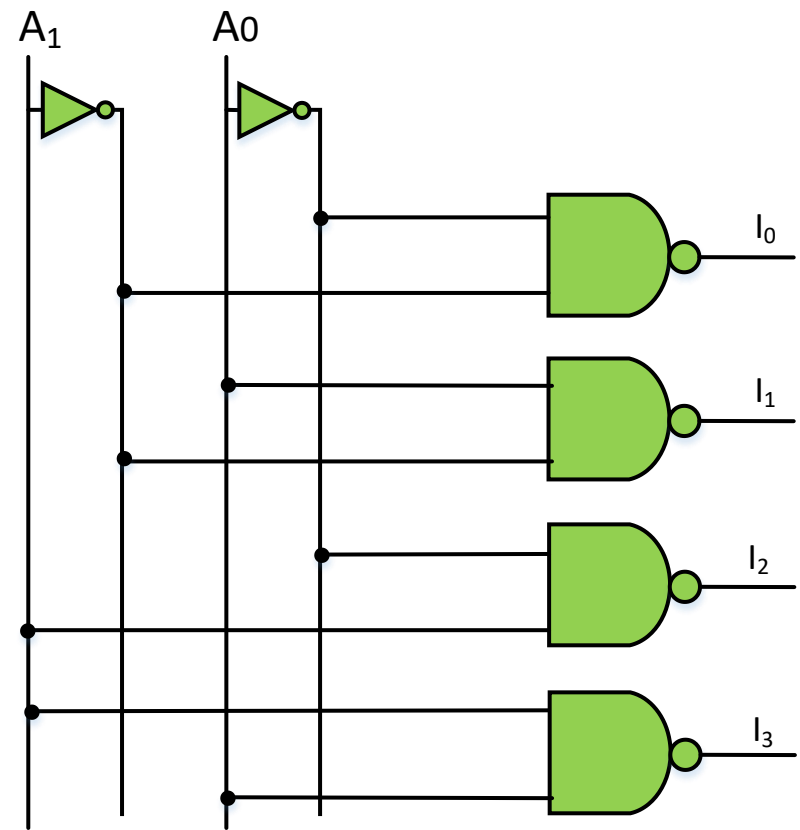
Combinational components

Decoder (functionality and gate level)



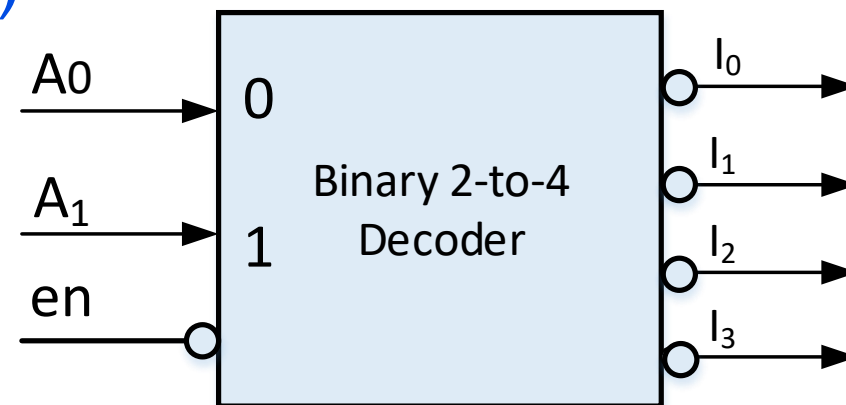
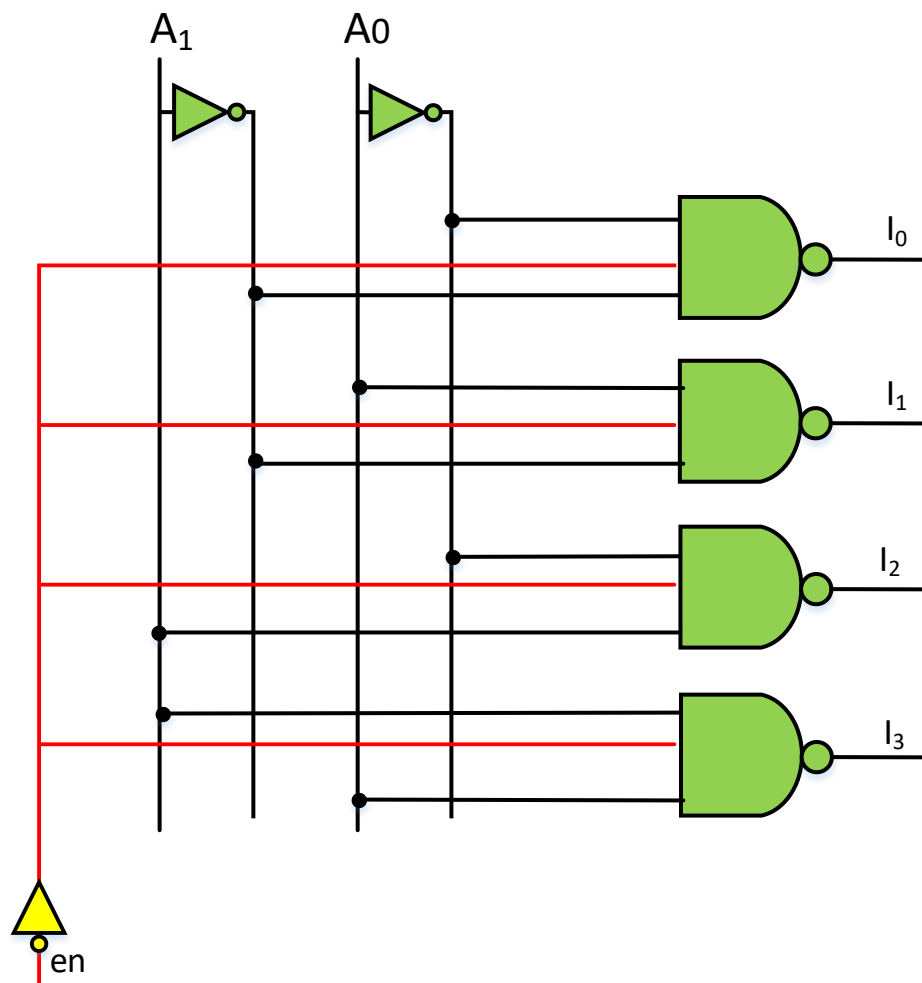
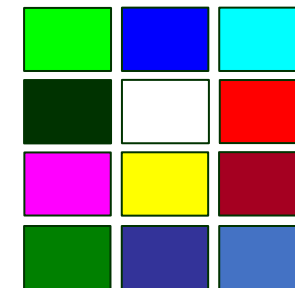
A_0	A_1	I_0	I_1	I_2	I_3
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

Active low outputs



Combinational components

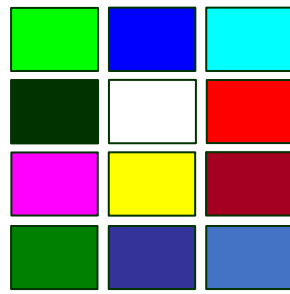
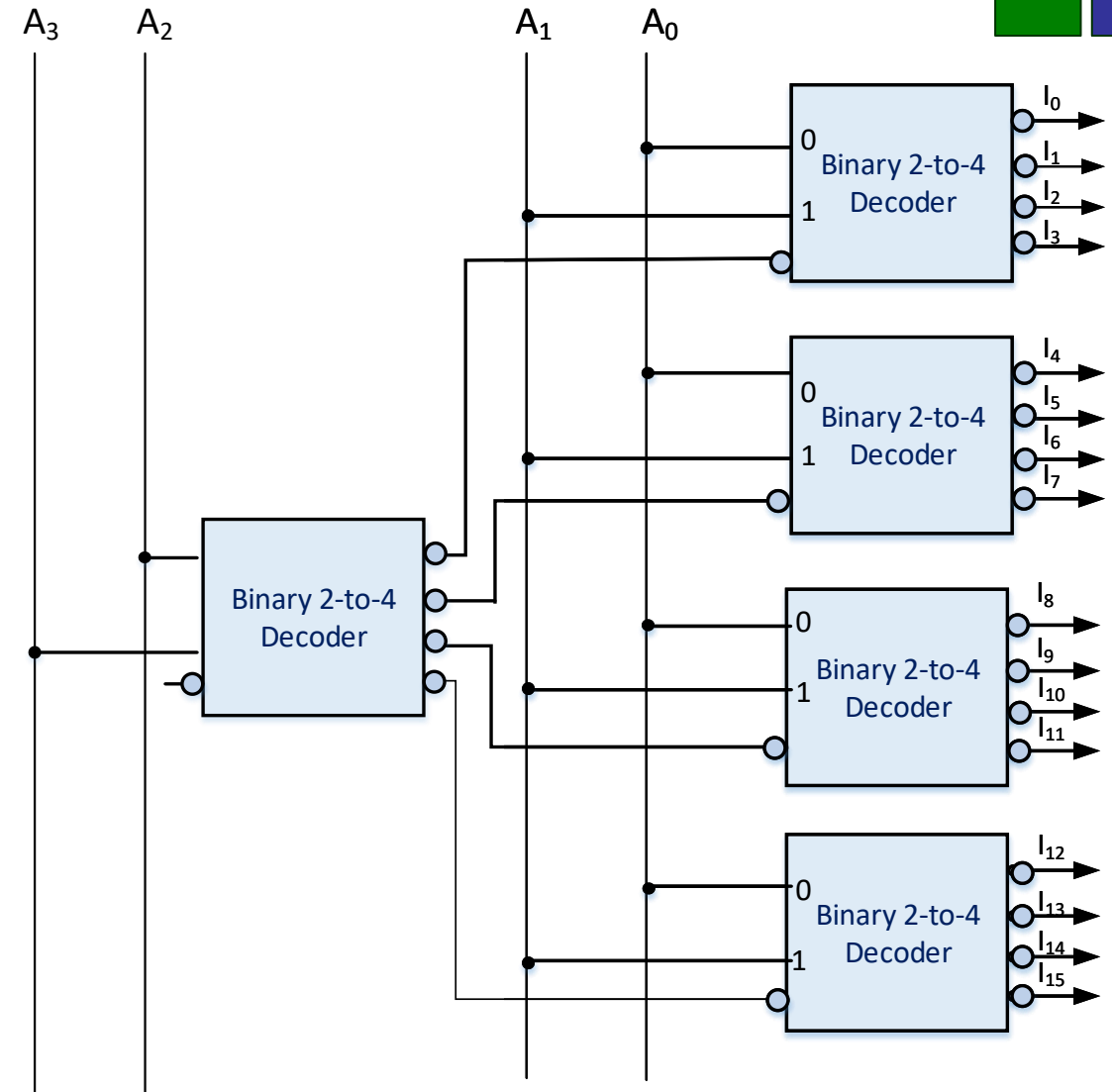
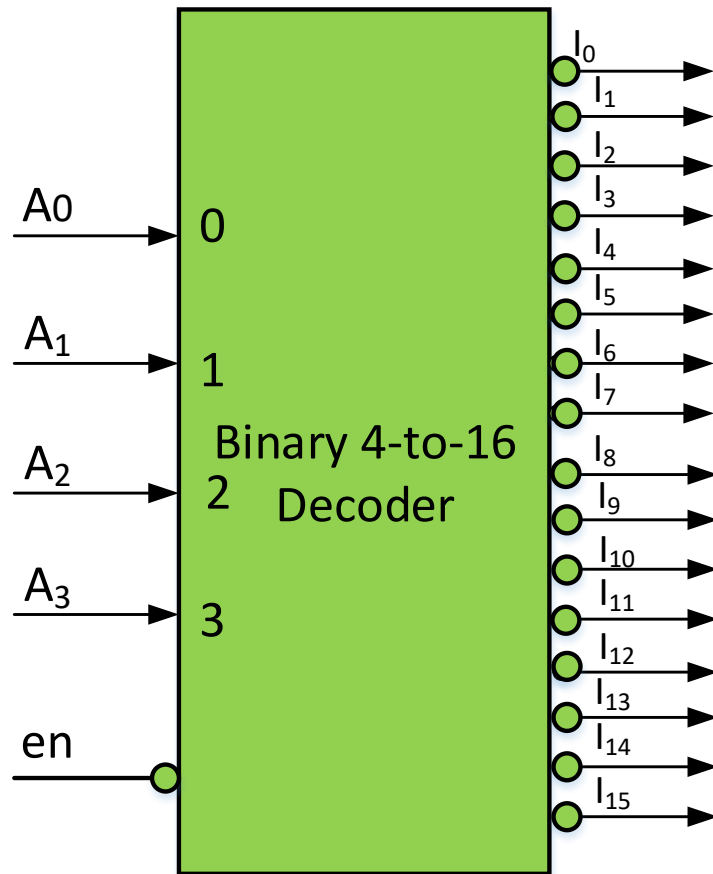
Decoder (functionality and gate level)

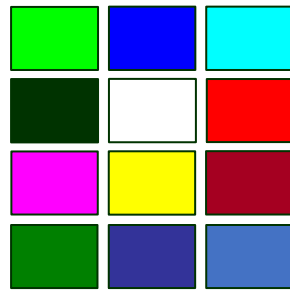


en	A ₀	A ₁	I ₀	I ₁	I ₂	I ₃
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0
1	-	-	1	1	1	1

Combinational components

Decoder (Cascading)





Combinational components

Decoder (SystemVerilog description)

- 4-state data type
- A hardware signal line

```
module dcd2to4_v1( input logic [1:0] A,  
                  input logic      en,  
                  output logic [0:3] I );  
  
    assign I = en ? 4'b1111 :  
                (A==2'b00) ? 4'b0111 :  
                (A==2'b01) ? 4'b1011 :  
                (A==2'b10) ? 4'b1101 :  
                (A==2'b11) ? 4'b1110 :  
                4'bxxxx;  
  
endmodule
```

$[0:3]I \equiv I_0I_1I_2I_3$

- conditional operator (?:)
- (condition) ? value_if_true : value_if_false;

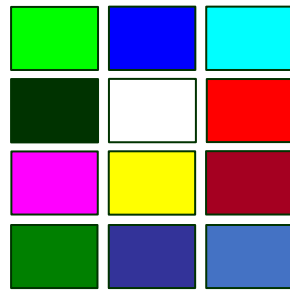
Combinational components

Decoder (SystemVerilog description)

```
module dcd2to4_v2 (  
    input  logic [1:0] A,  
    input  logic      en,  
    output logic [3:0] I  
);  
  
always_comb begin  
    I = 4'b1111;  
    if (en) begin  
        I = 4'b1111;  
    end else begin  
        case (A)  
            2'd0: I = 4'b1110;  
            2'd1: I = 4'b1101;  
            2'd2: I = 4'b1011;  
            2'd3: I = 4'b0111;  
            default: I = 4'bxxxx;  
        endcase  
    end  
end  
endmodule
```

$[3:0]I \equiv I_3I_2I_1I_0$

Output Rule



Combinational components

Decoder (SystemVerilog description)

Scalable Model

```
module dcd8to256 (  
    input logic [7:0] AA,  
    input logic      en,  
    output logic [0:255] II  
);  
  
    always_comb begin  
        II = '1;  
        if (!en) II[AA] = 1'b0;  
    end  
  
endmodule
```

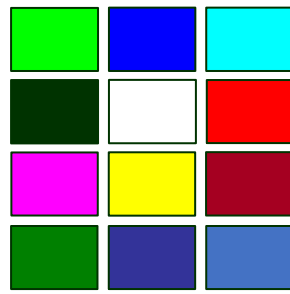
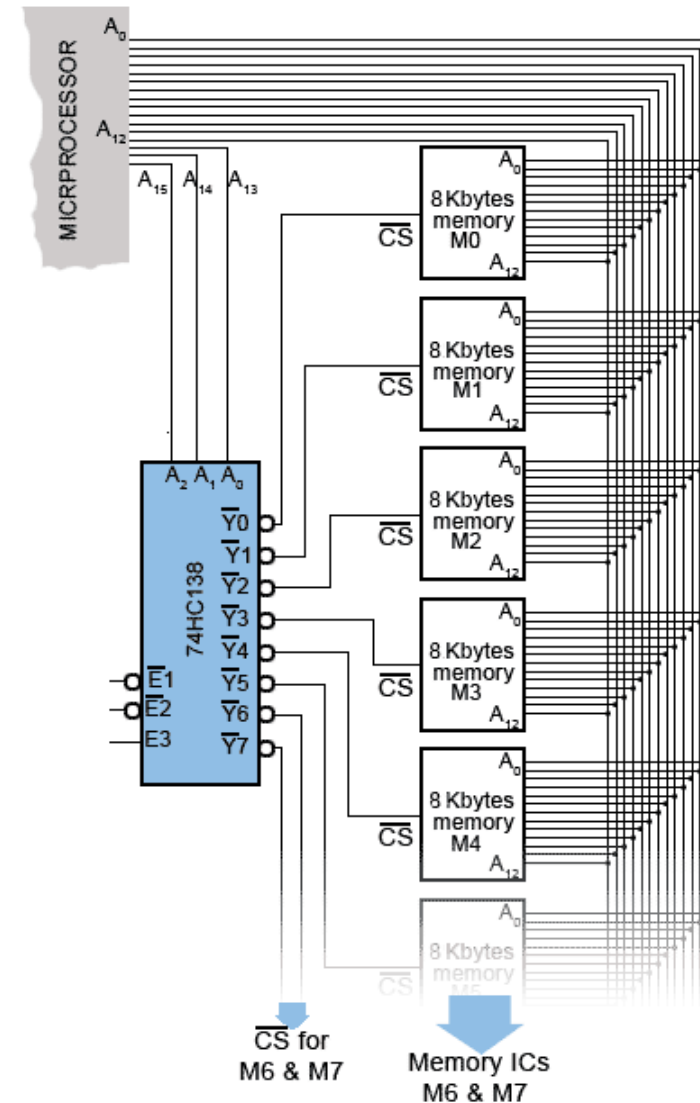
Can be replaced with
`II={256{1'b1}};`

`{}` : Replication operator

Combinational components

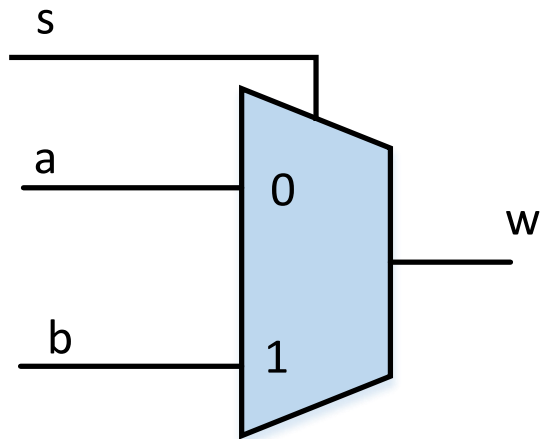
Decoder (Application)

Address decoding for memory

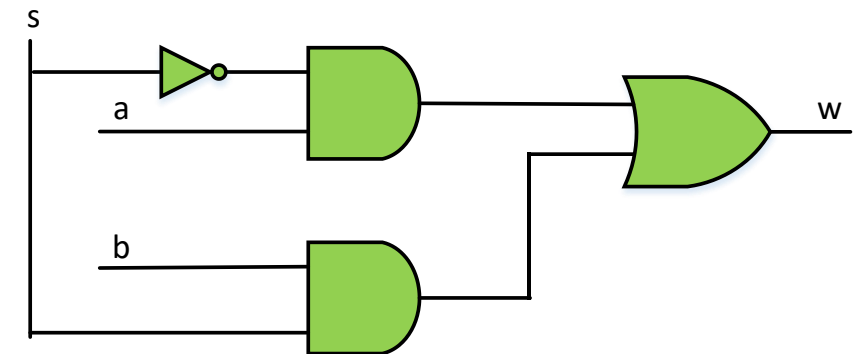


Combinational components

Multiplexer (Functionality and gate level)



S_1	S_0	w
0	0	a
0	1	b
1	0	c
1	1	d



$$w = \bar{s}.a + s.b$$

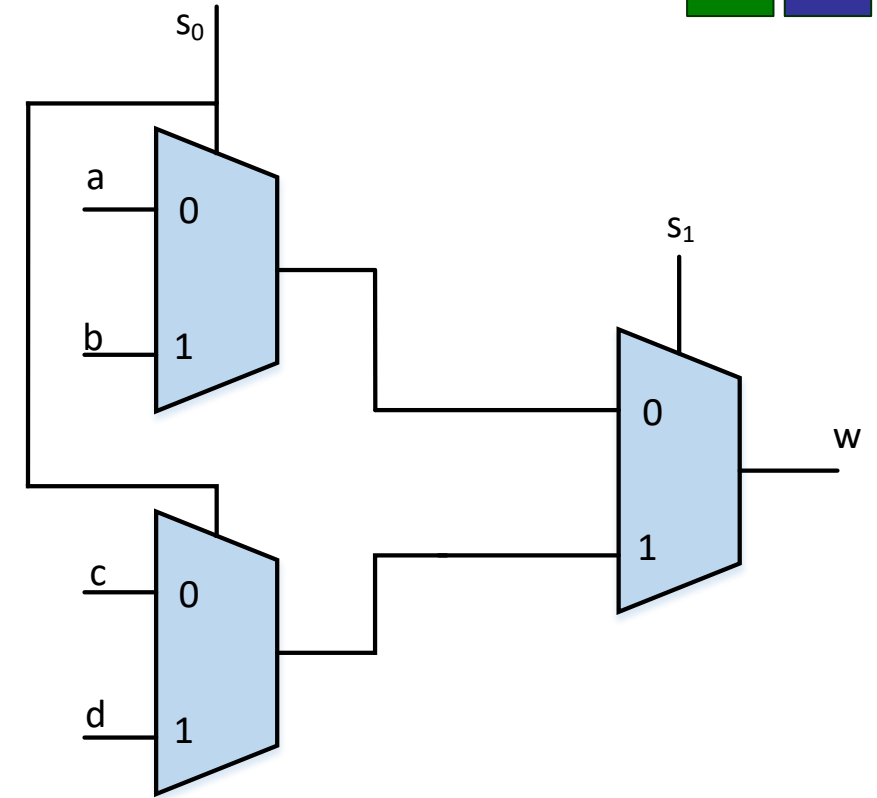
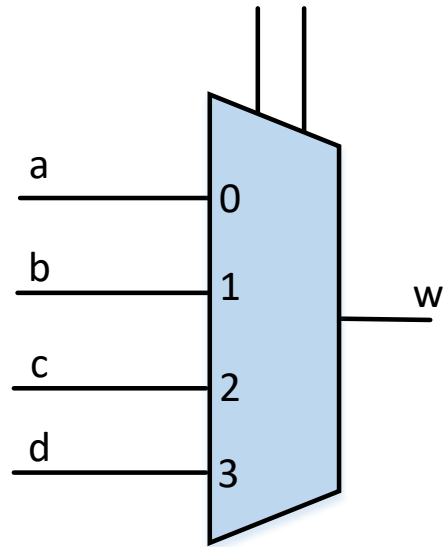
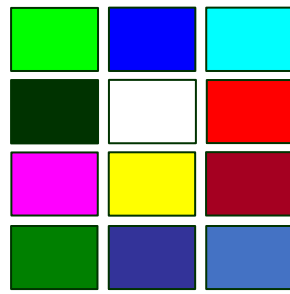
$$w = (\sim s \& a) | (s \& b)$$

$$w = \sim s ? a : b$$

Can be used to select between inputs
Can be used to implement arbitrary logic

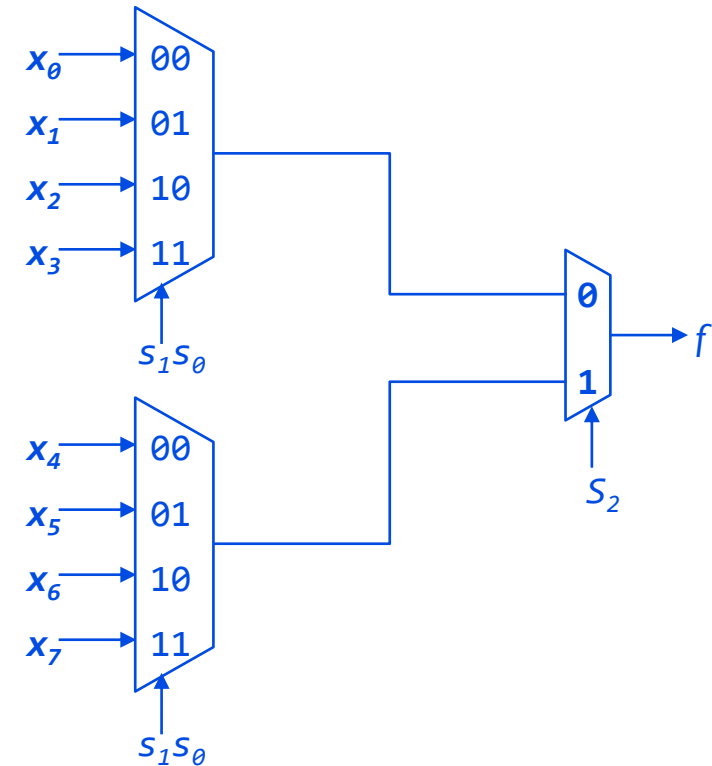
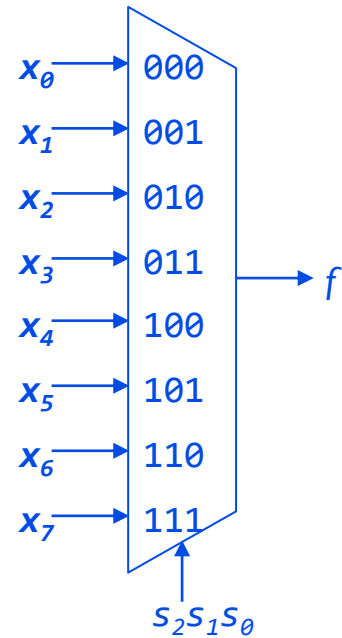
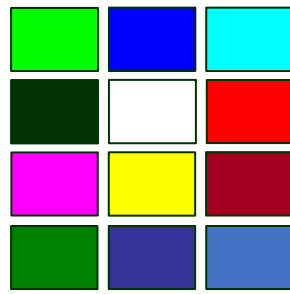
Combinational components

Multiplexer (Cascading)



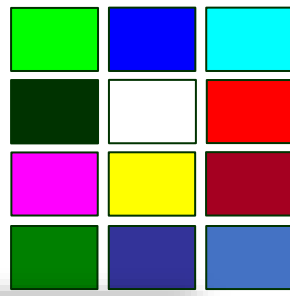
Combinational components

Multiplexer (Cascading)



Combinational components

Multiplexer (SystemVerilog description)



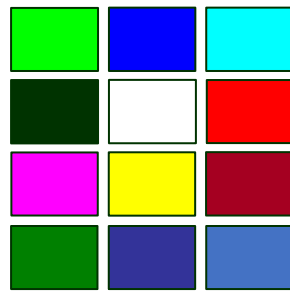
```
module mux4to1_v1 (  
    input  logic      a, b, c, d,  
    input  logic [1:0] s,  
    output logic      w  
);  
  
    assign w = (s == 2'b00) ? a :  
               (s == 2'b01) ? b :  
               (s == 2'b10) ? c :  
               (s == 2'b11) ? d : 1'bx;  
  
endmodule
```

```
module mux4to1_v2 (  
    input  logic      a, b, c, d,  
    input  logic [1:0] s,  
    output logic      w  
);  
  
    logic [3:0] J;  
    assign J = {d, c, b, a};  
    assign w = J[s];  
  
endmodule
```

```
module mux8to1 (  
    input  logic [7:0] J,  
    input  logic [2:0] s,  
    output logic      w  
);  
  
    assign w = J[s];  
  
endmodule
```

Combinational components

Multiplexer (SystemVerilog description)



Parameterized Mux

```
module mux_Nto1 #(
    parameter N = 4
) (
    input  logic [N-1:0] J,
    input  logic [$clog2(N)-1:0] s,
    output logic W
);

always_comb begin
    W = J[s];
end

endmodule
```

```
`timescale 1ns/1ps
module mux_Nto1_tb;

    localparam BW = 8;

    logic [BW-1:0] J;
    logic [$clog2(BW)-1:0] s;
    logic W;

    mux_Nto1 #(
        .N(BW)
    ) UUT (
        .J(J),
        .s(s),
        .W(W)
    );

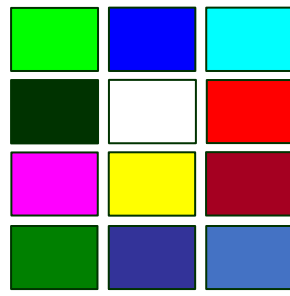
    initial begin
        J = $urandom;
        for (int i = 0; i < BW; i++) begin
            s = i;
            #7;
        end

        #12 $stop;
    end

endmodule
```

built-in functions used for generating random numbers

- \$urandom
- \$urandom_range(min, max)
- \$random
- \$random(seed)

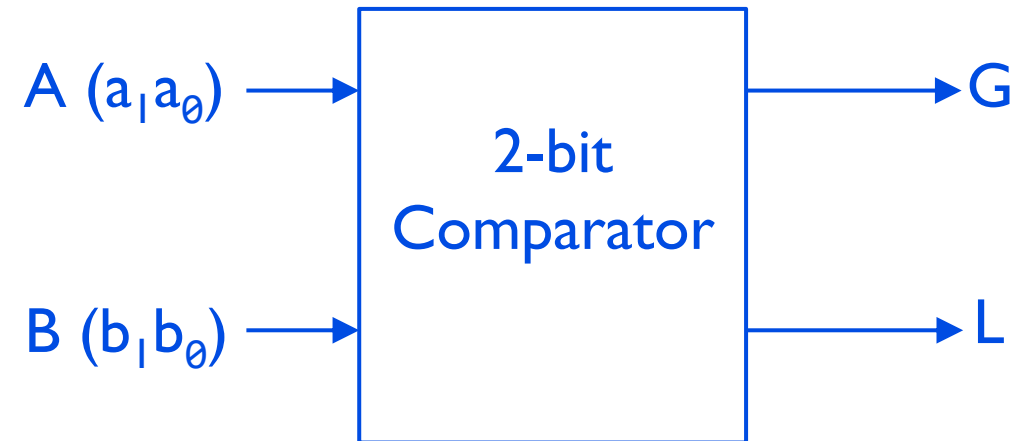


Combinational Components

Comparator

- A digital circuit that compares two 2-bit unsigned binary numbers

a_1	a_0	b_1	b_0	G	L
0	0	0	0	0	0
0	0	0	1	0	1
0	0	1	0	0	1
0	0	1	1	0	1
0	1	0	0	1	0
0	1	0	1	0	0
0	1	1	0	0	1
0	1	1	1	0	1
1	0	0	0	1	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	1	0	1
1	1	0	0	1	0
1	1	0	1	1	0
1	1	1	0	1	0
1	1	1	1	0	0

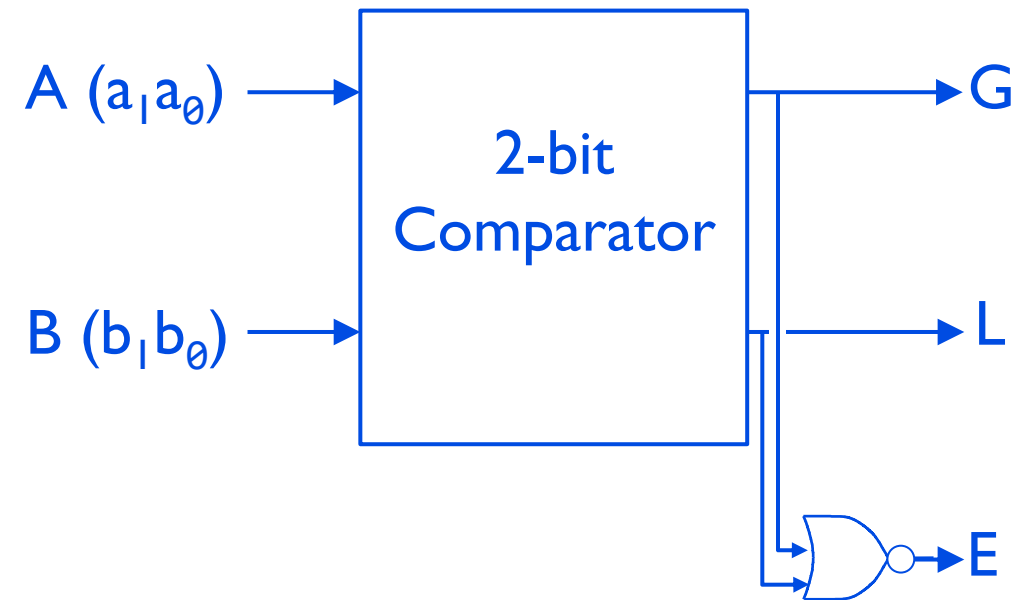


Combinational components

Comparator

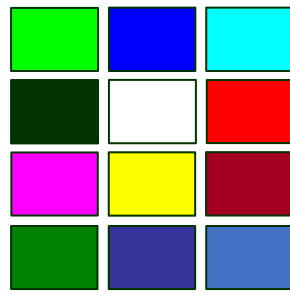
- A digital circuit that compares two 2-bit unsigned binary numbers

a_1	a_0	b_1	b_0	G	L	E
0	0	0	0	0	0	1
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	1	0	0
0	1	0	1	0	0	1
0	1	1	0	0	1	0
0	1	1	1	0	1	0
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	0	1
1	0	1	1	0	1	0
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	0	1



Combinational components

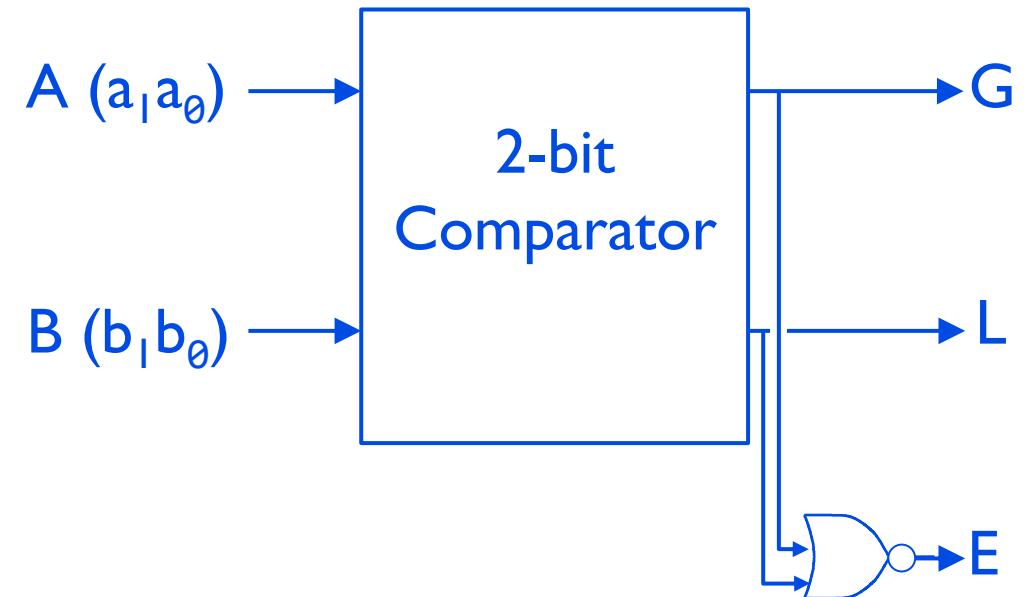
Comparator



$$G = a_1b_1' + a_0b_1'b_0' + a_1a_0b_0'$$

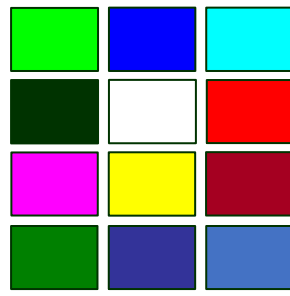
$$L = a_1'b_1 + a_1'a_0'b_0 + b_1b_0a_0'$$

$$E = (G+L)'$$



Combinational components

Comparator (SystemVerilog description)



$$G = a_1b_1' + a_0b_1'b_0' + a_1a_0b_0'$$

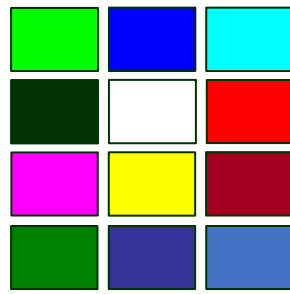
$$L = a_1'b_1 + a_1'a_0'b_0 + b_1b_0a_0'$$

$$E = (G+L)'$$

```
module comparator(  
    input logic [1:0] a,  
    input logic [1:0] b,  
    output logic E, G, L  
);  
  
    assign G = (a[1] & (~b[1])) |  
               (a[0] & (~b[0]) & (~b[1])) |  
               (a[0] & (~b[0]) & a[1]);  
  
    assign L = (b[1] & (~a[1])) |  
               (b[0] & (~a[0]) & (~a[1])) |  
               (b[0] & (~a[0]) & b[1]);  
  
    assign E = ~(G | L);  
  
endmodule
```

Combinational components

Comparator (SystemVerilog description)

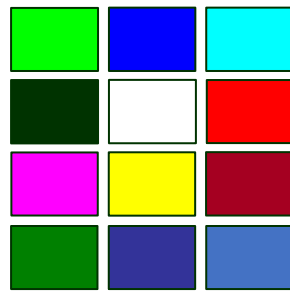


```
module cas4bitcmp_v1 (  
    input  logic [3:0] a, b,  
    output logic      L, E, G  
);  
  
    assign {L, E, G} = (a < b) ? 3'b100 :  
                      (a > b) ? 3'b001 :  
                      (a == b) ? 3'b010 : 3'bxxx;
```

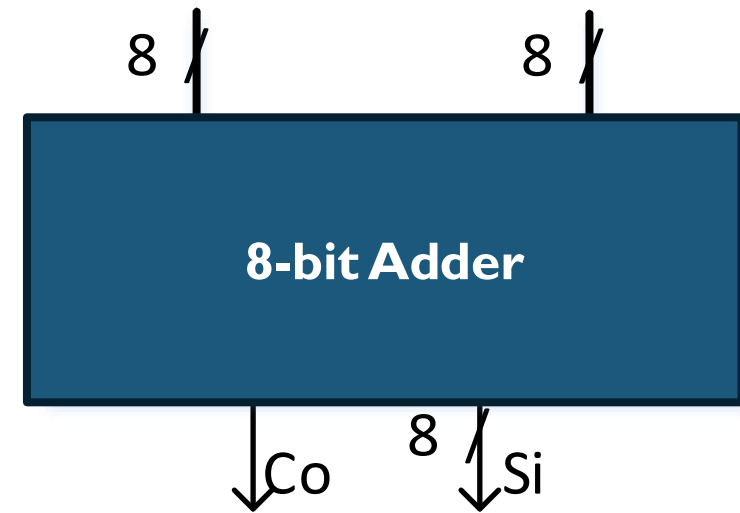
```
module cas4bitcmp_v2 (  
    input  logic [3:0] a, b,  
    output logic      L, E, G  
);  
  
    always_comb begin  
        {L, E, G} = 3'b000;  
        if (a < b)      L = 1'b1;  
        else if (a > b)  G = 1'b1;  
        else if (a == b) E = 1'b1;  
    end  
  
endmodule
```

Combinational components

Adder (Functionality and gate level)

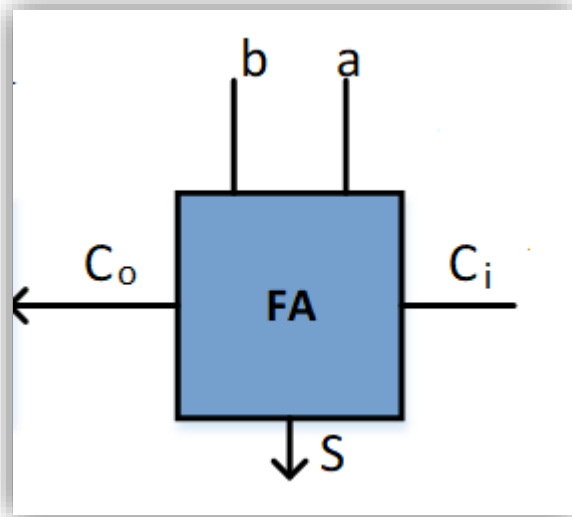
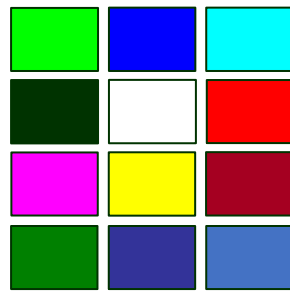


$$\begin{array}{r} 10110101 \\ + 00101101 \\ \hline 011100010 \end{array}$$



Combinational components

Adder (Functionality and gate level)



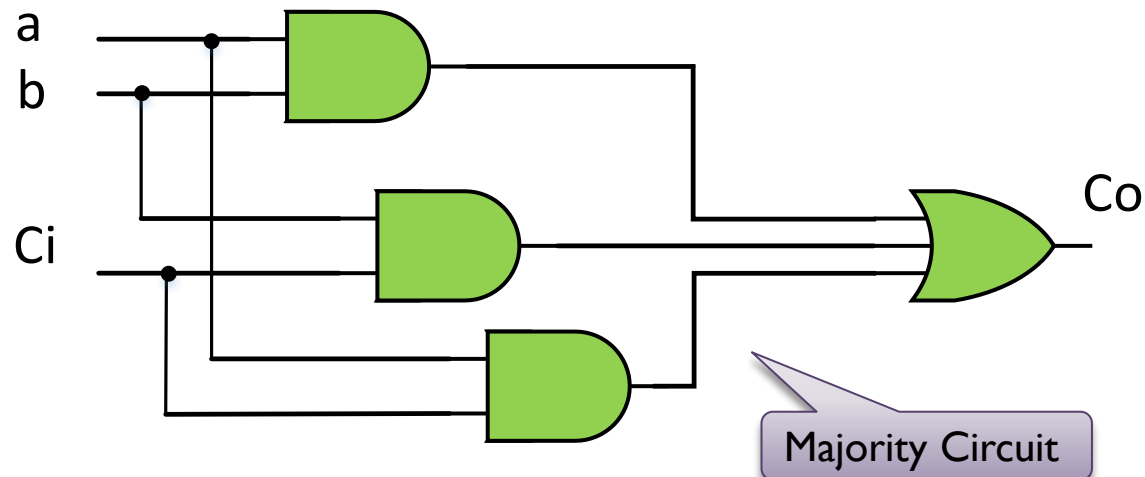
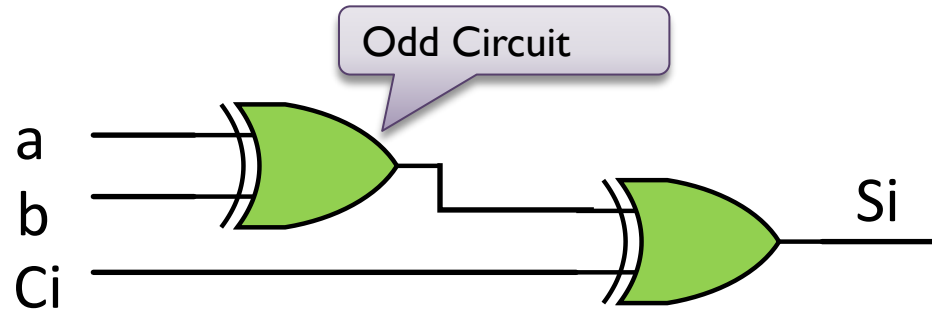
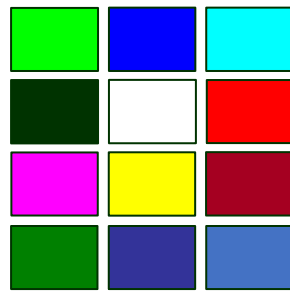
abc _i	C _o S
000	00
001	01
010	01
011	10
100	01
101	10
110	10
111	11

ab \ ci	00	01	11	10
0	0 0	1 2	0 6	1 4
1	1 1	0 3	1 7	0 5

ab \ ci	00	01	11	10
0	0 0	0 2	1 6	0 4
1	0 1	1 3	1 7	1 5

Combinational components

Adder (Functionality and gate level)



ab \ ci	00	01	11	10
0	0 0	1 2	0 6	1 4
1	1 1	0 3	1 7	0 5

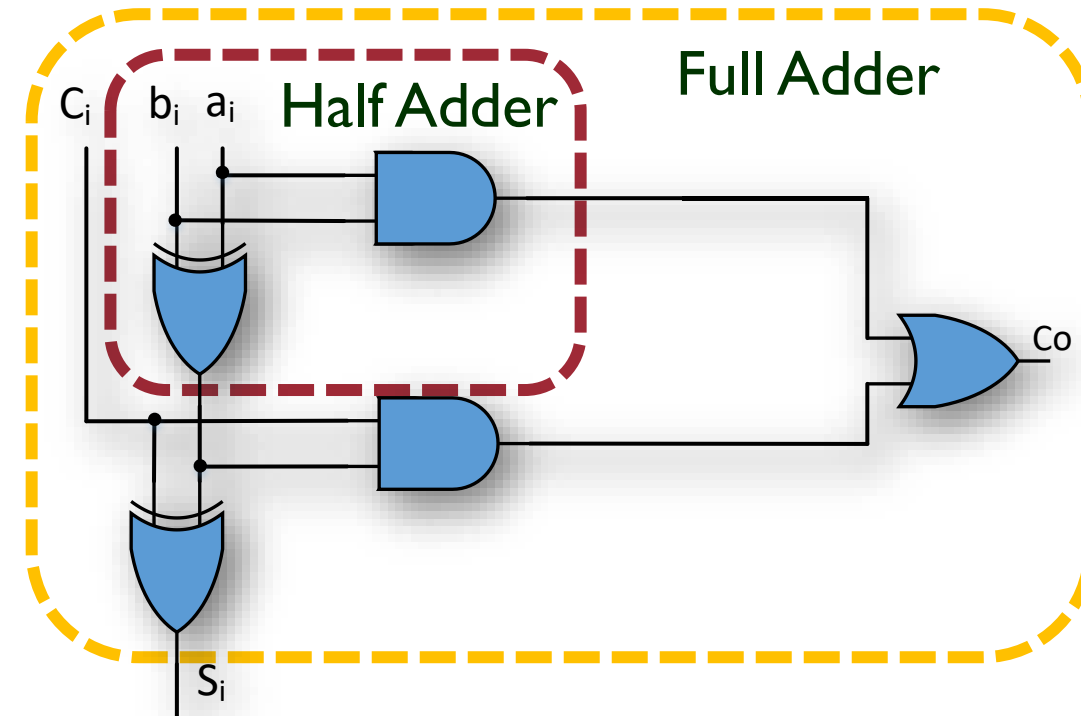
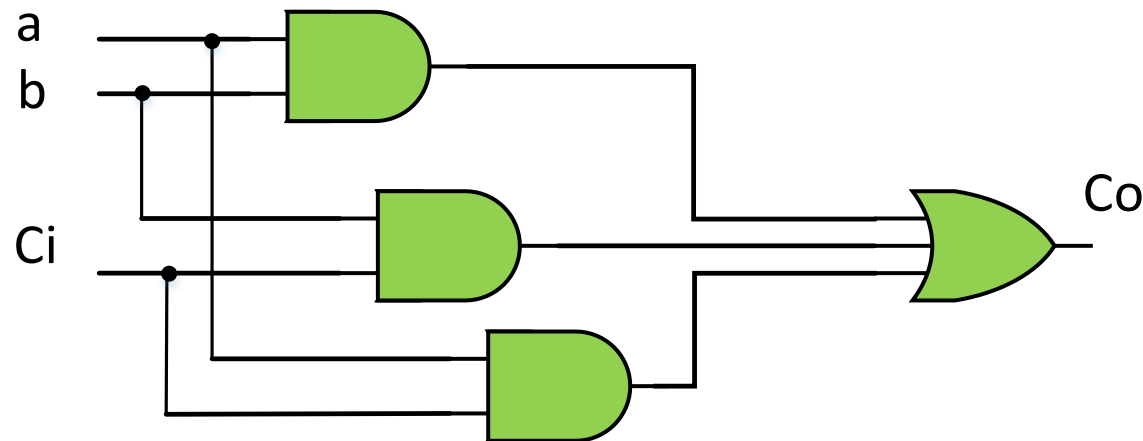
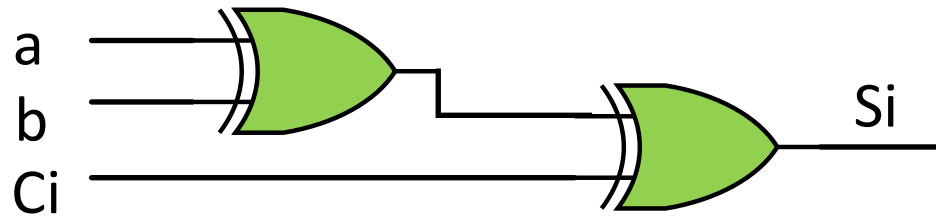
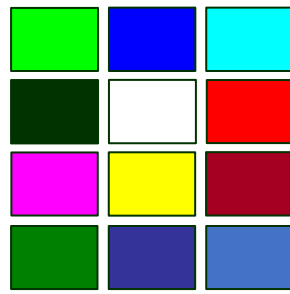
s

ab \ ci	00	01	11	10
0	0 0	0 2	1 6	0 4
1	0 1	1 3	1 7	1 5

co

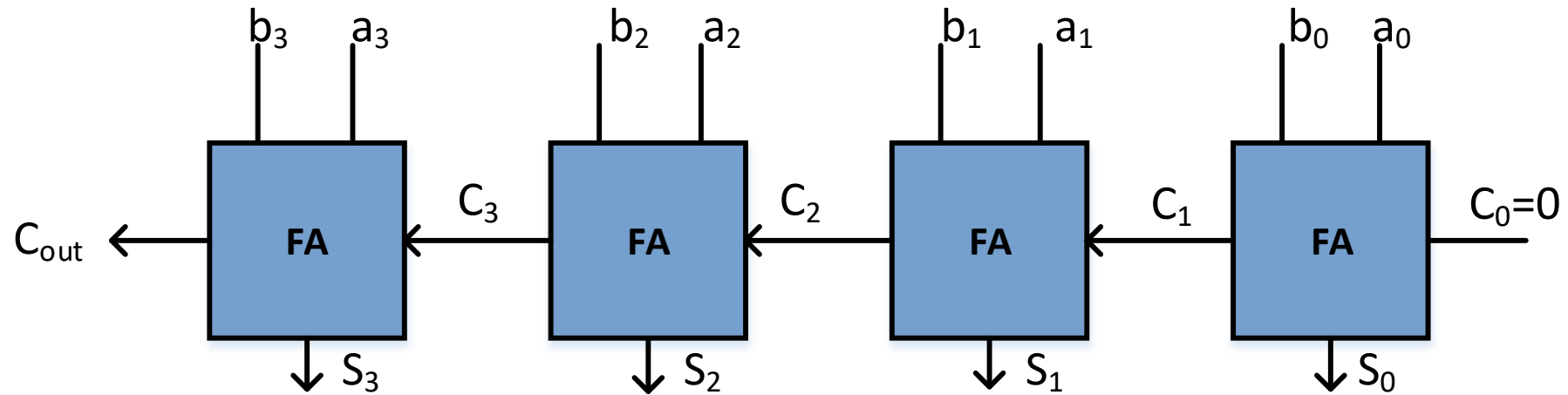
Combinational components

Adder (Functionality and gate level)

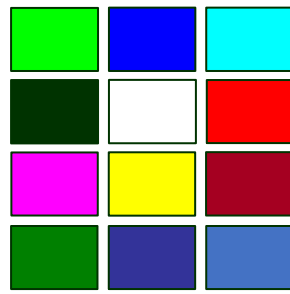
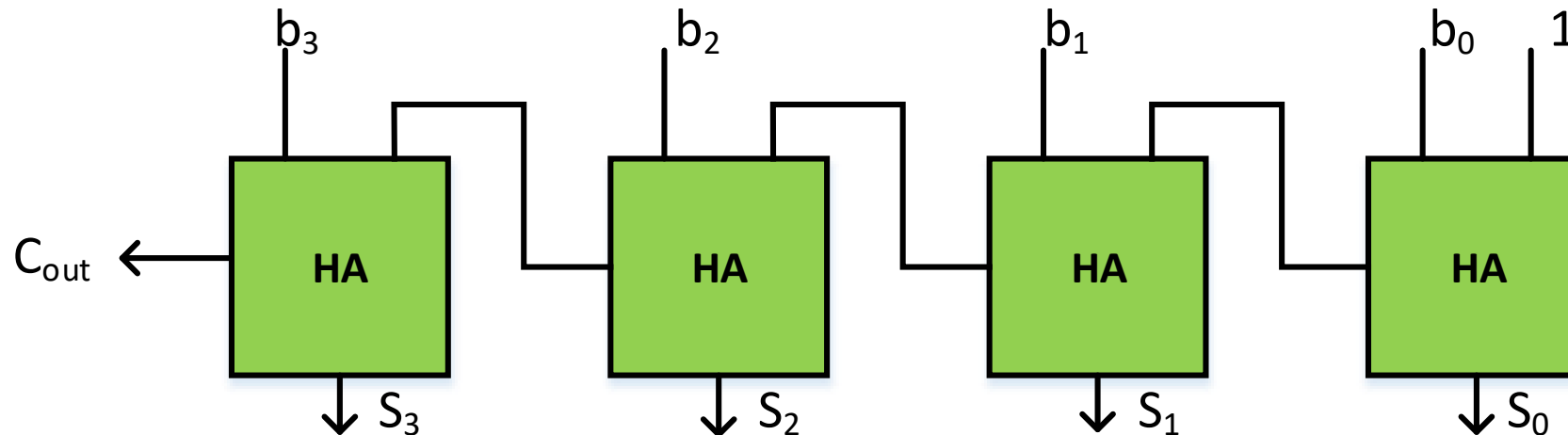


Combinational components

Ripple Carry Adder

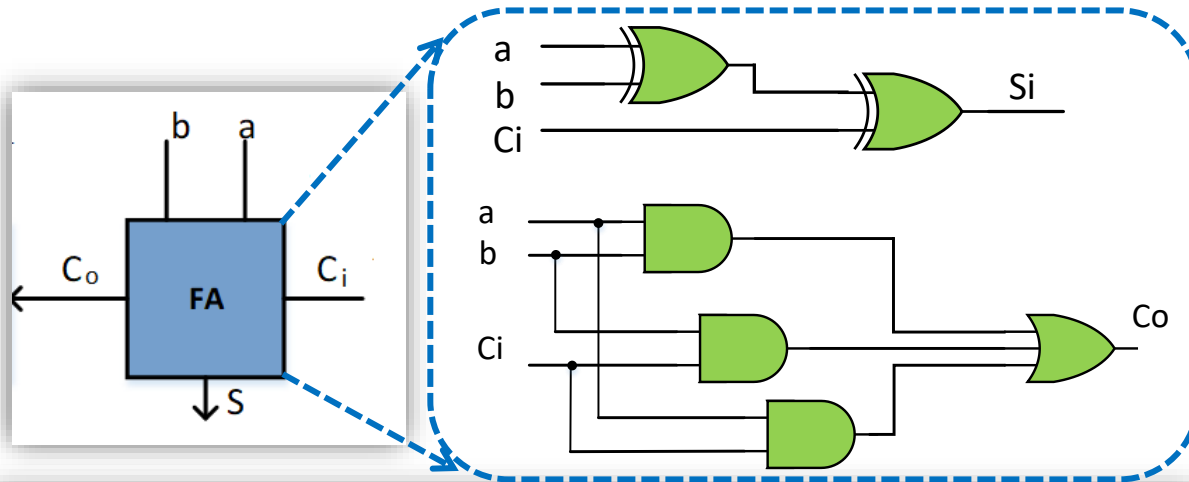
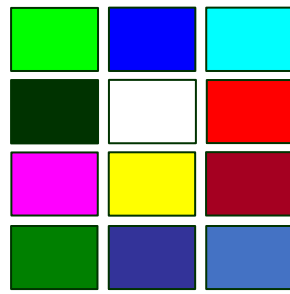


Incrementer

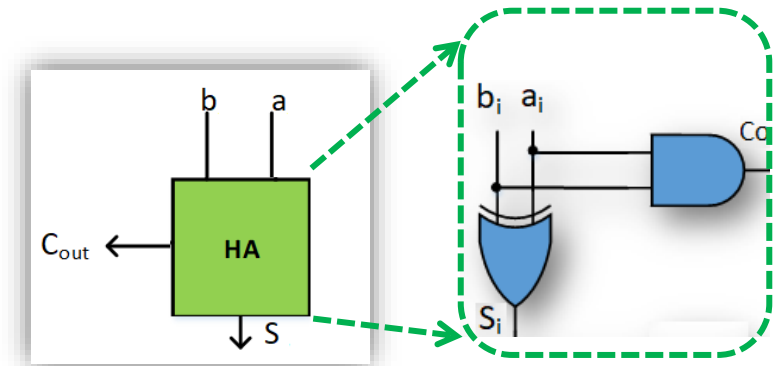


Combinational components

Full adder (SystemVerilog description)



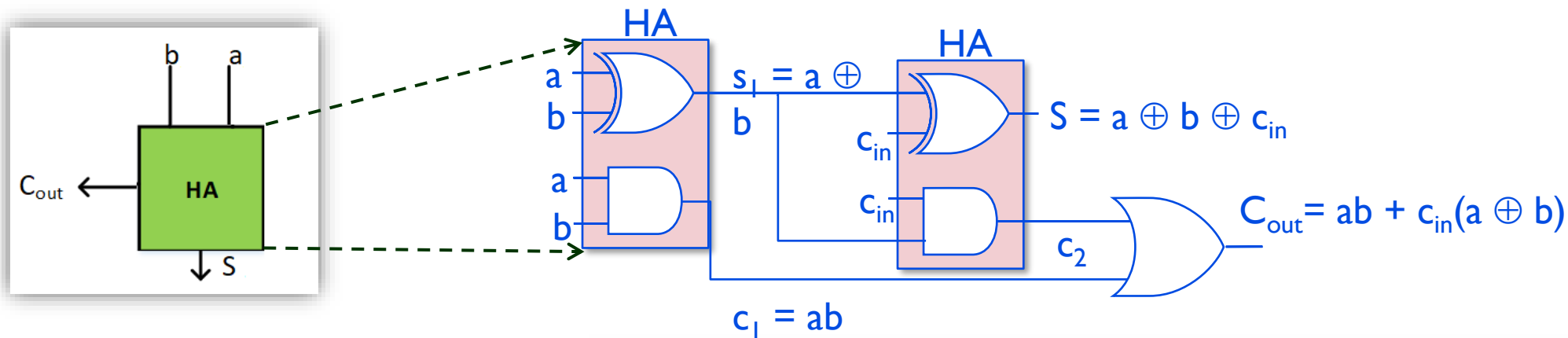
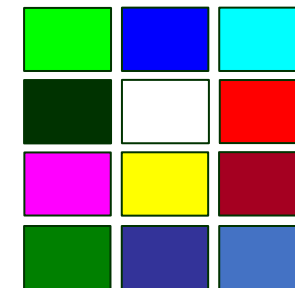
```
module FA_v2 (  
    input logic a, b, cin,  
    output logic S, Co  
);  
  
    assign S = a ^ b ^ cin;  
    assign Co = (a & b) | (a & cin) | (b & cin);  
  
endmodule
```



```
module half_adder (  
    input logic a,b,  
    output logic c_out, s  
);  
  
    assign s = a^b;  
    assign c_out = a&b;  
endmodule
```

Combinational components

Full Adder in terms of HA (SystemVerilog description)



```

module half_adder (
    input logic a,b,
    output logic c_out, s
);
    assign s = a^b;
    assign c_out = a&b;
endmodule

```

```

module FA_byHAs (
    input logic a, b, c_in,
    output logic c_out, s
);
    logic s1,c1,c2;

    half_adder ha1(.a(a),.b(b),.s(s1),.c_out(c1));
    half_adder ha2(s1,c_in,c2,s);
    assign c_out = c1|c2;

endmodule

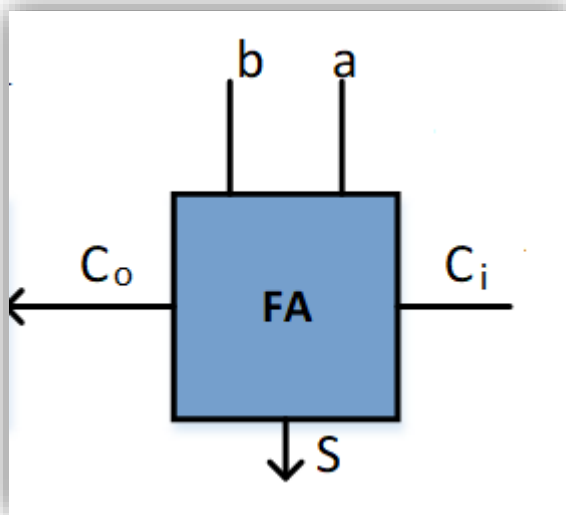
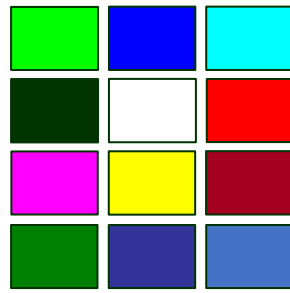
```

Named port association

Positional port association

Combinational components

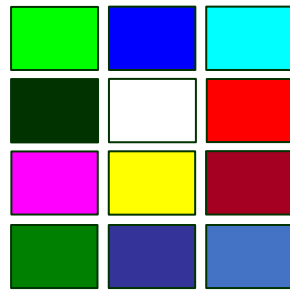
Full adder (SystemVerilog description)



```
module FA_v1 (  
    input  logic a, b, cin,  
    output logic S, Co  
);  
  
    assign {Co, S} = a + b + cin;  
  
endmodule
```

Combinational components

8-bit adder (SystemVerilog description)

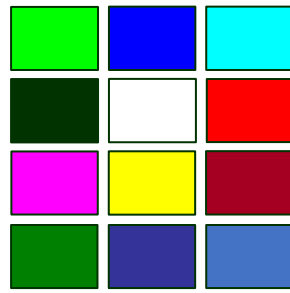


```
module add8_v1 (  
    input  logic [7:0] A, B,  
    input  logic      Cin,  
    output logic [7:0] S,  
    output logic      Co  
);  
  
    assign {Co, S} = A + B + Cin;  
  
endmodule
```

```
module add8_v2 (  
    input  logic [7:0] A, B,  
    input  logic      Cin,  
    output logic [7:0] S,  
    output logic      Co  
);  
  
    always_comb begin  
        {Co, S} = A + B + Cin;  
    end  
  
endmodule
```


Combinational components

8-bit Incrementer (SystemVerilog description)



```
module Inc8 (  
    input  logic [7:0] A,  
    output logic [7:0] S,  
    output logic      Co  
);  
  
    assign {Co, S} = A + 1;  
  
endmodule
```

Combinational components

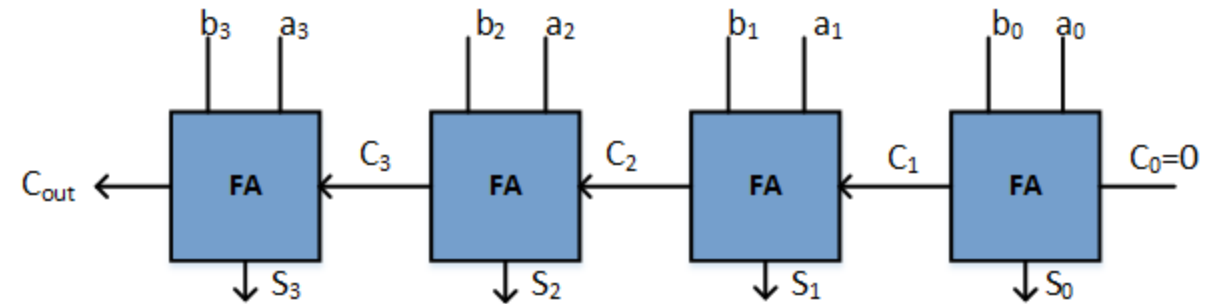
4-bit adder (SystemVerilog description)

An iterative structure

```
module add_Nbit_gen #(
    parameter int SIZE
) (
    input  logic [SIZE-1:0] a, b,
    input  logic          ci,
    output logic [SIZE-1:0] s,
    output logic          co
);
    logic [SIZE:0] c;

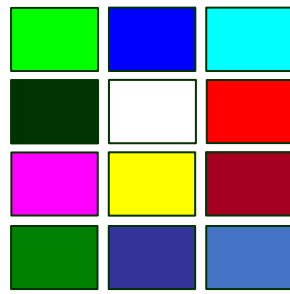
    assign c[0] = ci;
    assign co = c[SIZE];

    genvar i;
    generate
        for (i = 0; i < SIZE; i = i + 1) begin : full_adders
            FA_v1 fa (a[i], b[i], c[i], s[i], c[i+1]);
        end
    endgenerate
endmodule
```



Combinational components

4-bit adder (SystemVerilog description)



```
module add_Nbit_gen_tb;
```

```
    localparam SIZE = 4;
```

```
    logic [SIZE-1:0] a, b;
```

```
    logic ci;
```

```
    logic [SIZE-1:0] s;
```

```
    logic co;
```

```
    add_Nbit_gen #(.SIZE(SIZE)) dut (
        .a(a),
        .b(b),
        .ci(ci),
        .s(s),
        .co(co)
    );
```

```
    initial begin
```

```
        $display("Time\t a\t b\t ci\t => s\t co");
```

```
        $display("-----");
```

```
        for (int i = 0; i < 10; i++) begin
```

```
            a = $urandom_range(0, 15);
```

```
            b = $urandom_range(0, 15);
```

```
            ci = $urandom_range(0, 1);
```

```
            #1;
```

```
            $display("%t\t%d\t%d\t%d\t => %d\t%d", $time, a, b, ci, s, co);
```

```
        end
```

```
        $finish;
```

```
    end
```

```
endmodule
```

Combinational components

8-bit signed adder (SystemVerilog description)

- Signed Arithmetic uses sign-extended operands and signed interpretation
- Unsigned Arithmetic uses zero-extended operands and unsigned interpretation

All operands signed → operation is signed

Any operand unsigned → operation is unsigned

SystemVerilog uses **two's complement** to represent signed numbers

```
module signed_adder (  
    input  logic signed [7:0] a_signed,  
    input  logic signed [7:0] b_signed,  
    input  logic          [7:0] a_unsigned,  
    input  logic          [7:0] b_unsigned,  
    output logic signed [8:0] sum_signed,  
    output logic          [8:0] sum_unsigned  
);  
  
    assign sum_signed    = a_signed + b_signed;  
    assign sum_unsigned = a_unsigned + b_unsigned;  
  
endmodule
```

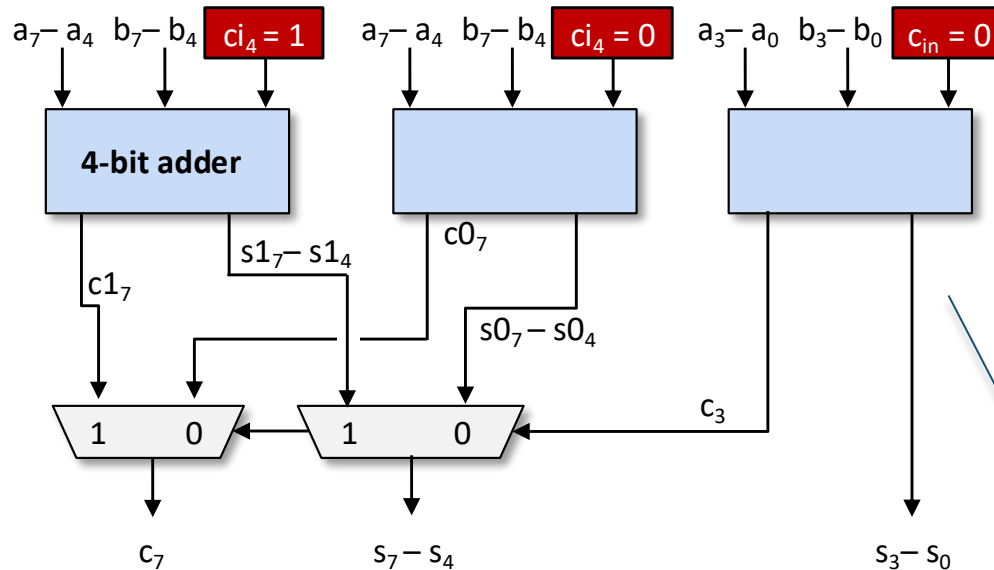
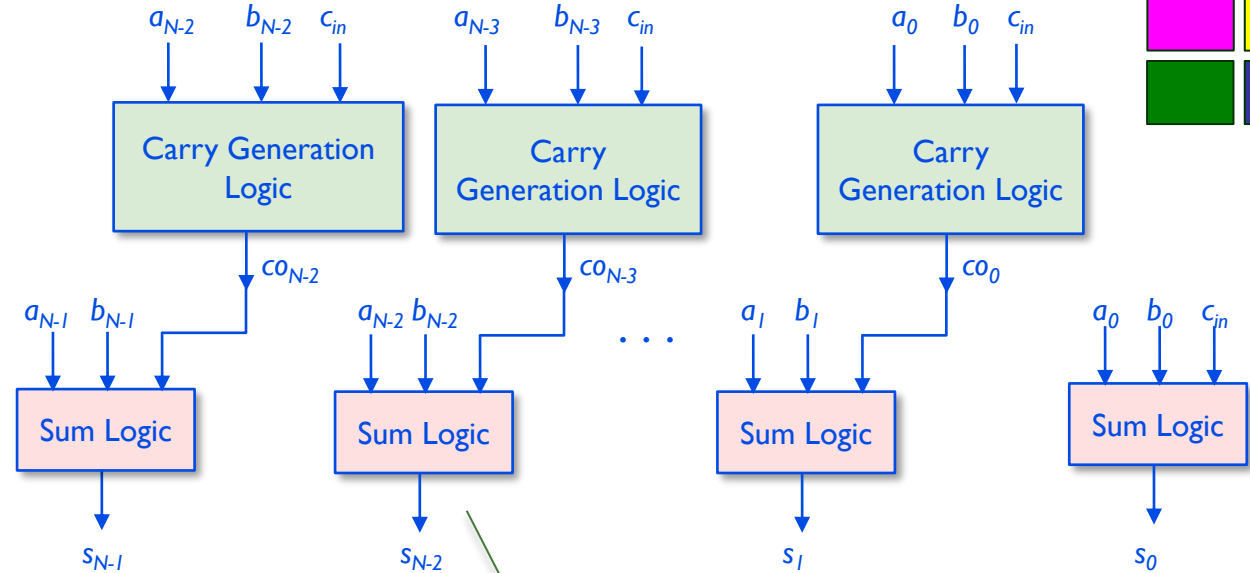
	Msgs			
+ /signed_adder_tb/a_s	68	68	100	-30
+ /signed_adder_tb/b_s	-80	-80	60	-50
+ /signed_adder_tb/sum_s	-12	-12	160	-80
+ /signed_adder_tb/a_u	68	68	100	200
+ /signed_adder_tb/b_u	176	176	60	100
+ /signed_adder_tb/sum_u	244	244	160	300

	Msgs			
+ /signed_adder_tb/a_s	01000100	01000100	01100100	11100010
+ /signed_adder_tb/b_s	10110000	10110000	00111100	11001110
+ /signed_adder_tb/sum_s	111110100	111110100	010100000	110110000
+ /signed_adder_tb/a_u	01000100	01000100	01100100	11001000
+ /signed_adder_tb/b_u	10110000	10110000	00111100	01100100
+ /signed_adder_tb/sum_u	011110100	011110100	010100000	100101100

Combinational components

Other Types of Adder

- Ripple Carry Adder
- Carry Look-ahead Adder
- Carry Select Adder
- ...



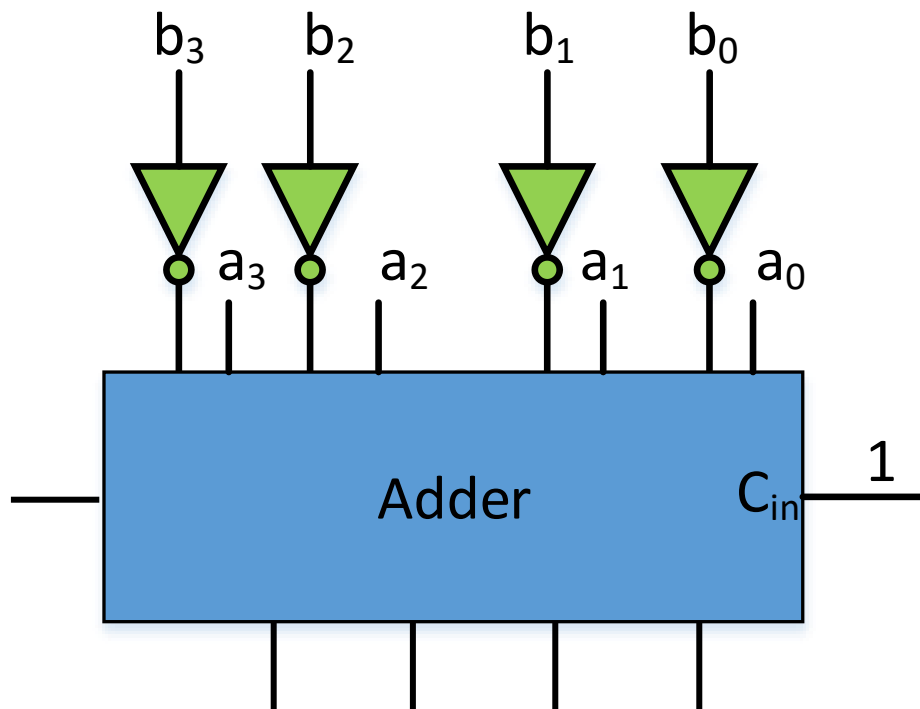
Carry Look Ahead adder
Precomputes and generates carry signals
based on input and carry-in signals

Carry Select adder employs separate adders
in parallel

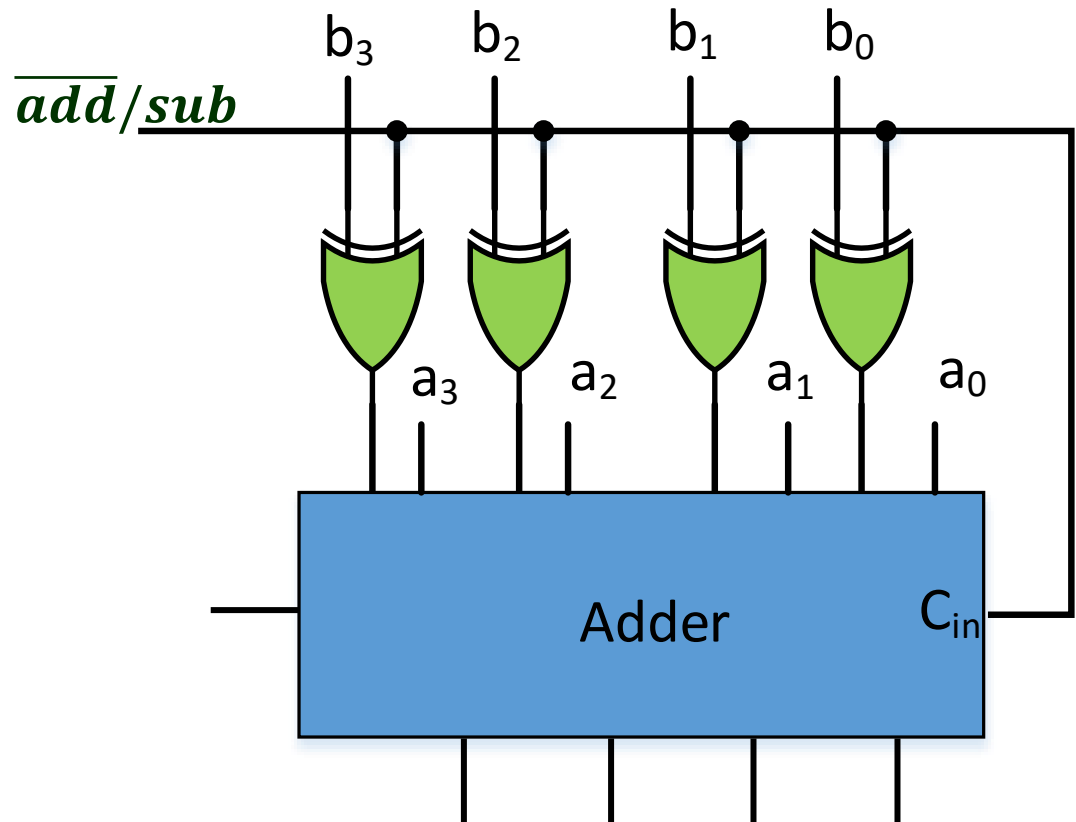
Combinational components

Adder / Subtractor

We can perform subtraction by simply inverting the bits of one operand and setting the carry in bit of the operation to 1.



$$a - b = a + (-b) = a + (\bar{b} + 1)$$



Combinational components

Shifter (functionality)

Shift n bits left: Multiplication by 2^n
Shift n bits right: Division by 2^n

- **Logical shift:** shifts the number to the left or right and fills the empty place with zero.



- **Arithmetic shift:** Is like the logical shift but replicates the leftmost bit (sign bit) when performing right shift. This is **signing extension**.

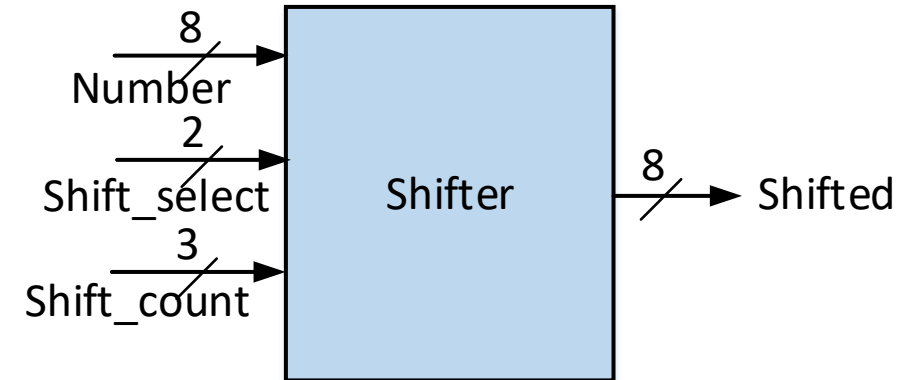


Combinational components

Shifter (Verilog description)

For Arithmetic Shifting

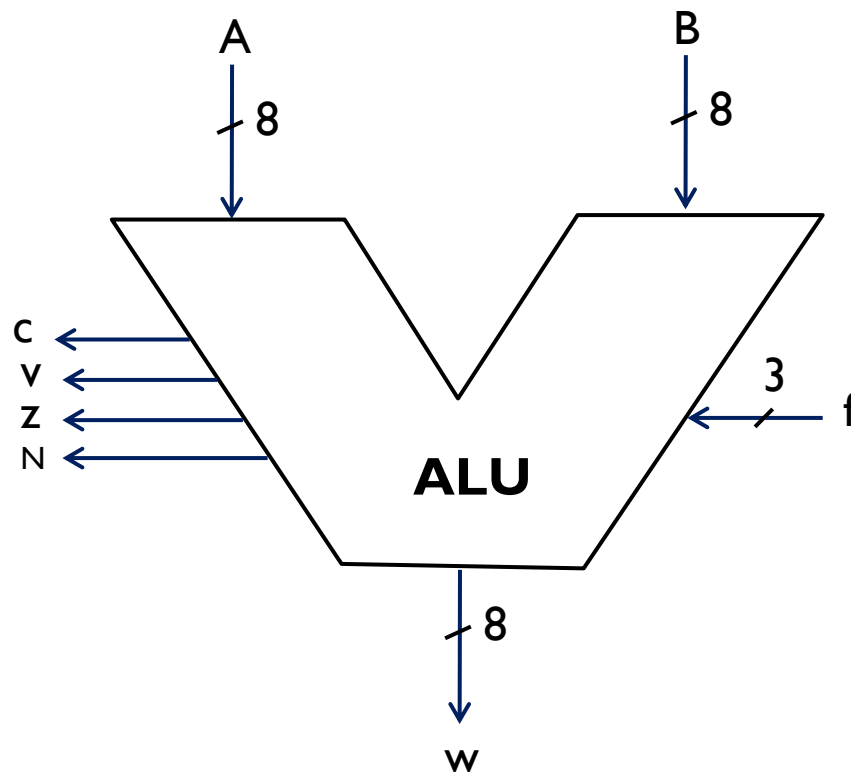
```
module shifter (  
    input logic signed [7:0] number,  
    input logic [1:0] shift_select,  
    input logic [2:0] shift_count,  
    output logic signed [7:0] out  
);  
  
always_comb begin  
    out = '0;  
    case (shift_select)  
        2'b00: out = number >> shift_count;  
        2'b01: out = number << shift_count;  
        2'b10: out = number >>> shift_count;  
        2'b11: out = number <<< shift_count;  
        default: out = '0;  
    endcase  
end  
endmodule
```



Combinational components

ALU (Arithmetic Logic Unit)

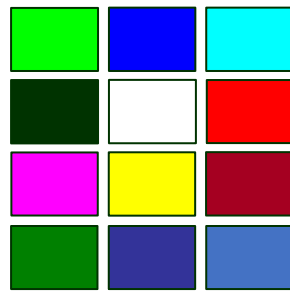
- An ALU is a component that can do both arithmetic and logic operations on its inputs.
- It can also have output flags such as overflow flag, zero flag, etc, as well as its operation results.



f_2	f_1	f_0	w
0	0	0	A+B
0	0	1	A*2
0	1	0	Max(A,B)
0	1	1	A&B
1	0	0	A B
1	0	1	2's complement of input A
1	1	0	A
1	1	1	$\sim A$

Combinational components

ALU (SystemVerilog description)

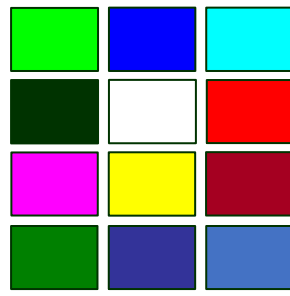


```
module ALU8 (  
    input logic signed [7:0] A, B,  
    input logic          [2:0] f,  
    output logic signed [7:0] w,  
    output logic         c, V,  
    output logic         Z, N  
);  
  
    always_comb begin  
        w = 8'b0;  
        c = 1'b0;  
        case (f)  
            3'd0: {c, w} = A + B;  
            3'd1: {c, w} = {A, 1'b0};  
            3'd2: w      = (A > B) ? A : B;  
            3'd3: w      = A & B;  
            3'd4: w      = A | B;  
            3'd5: w      = ~A + 1'b1;  
            3'd6: w      = A;  
            3'd7: w      = ~A;  
            default: w    = 8'd0;  
        endcase  
    end
```

```
        always_comb begin  
            V = 1'b0;  
            if (f == 3'b000) begin  
                if ( ( A[7] & B[7] & ~w[7] ) |  
                    ( ~A[7] & ~B[7] & w[7] ) )  
                    V = 1'b1;  
            end  
            else if (f == 3'b001) begin  
                V = A[7] ^ A[6];  
            end  
            else  
                V = 1'b0;  
            end  
  
            assign N = w[7];  
            assign Z = ~|w;  
        endmodule
```

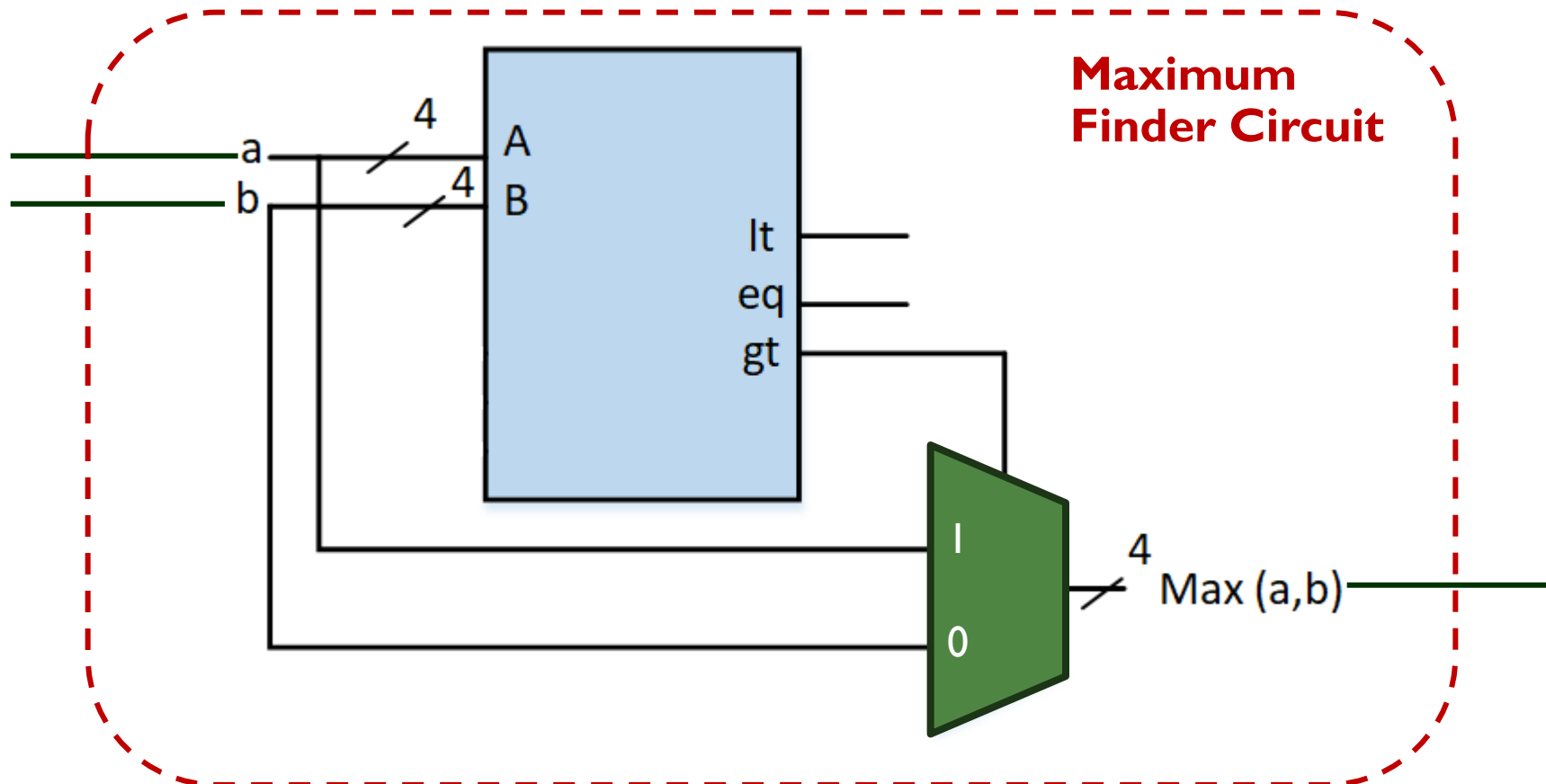
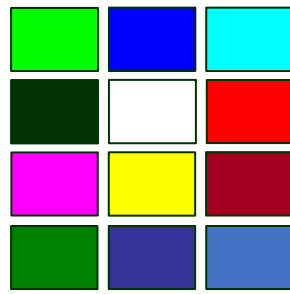
Combinational Components

- Decoder
- Multiplexer
- Comparator
- Adder
- Shifter
- ALU
- **Case Study**
 - **Max Finder**
 - **One's Counter**



Combinational components

Case Study (Maximum Finder)

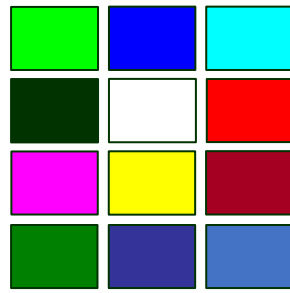


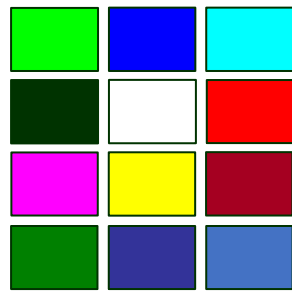
Combinational components

Case Study (Maximum Finder)

SystemVerilog description

```
module MaxFinder (  
    input  logic [3:0] A, B,  
    output logic [3:0] w  
);  
  
    logic gt;  
    assign gt = (A > B) ? 1'b1 : 1'b0;  
    assign w  = gt ? A : B;  
  
endmodule
```

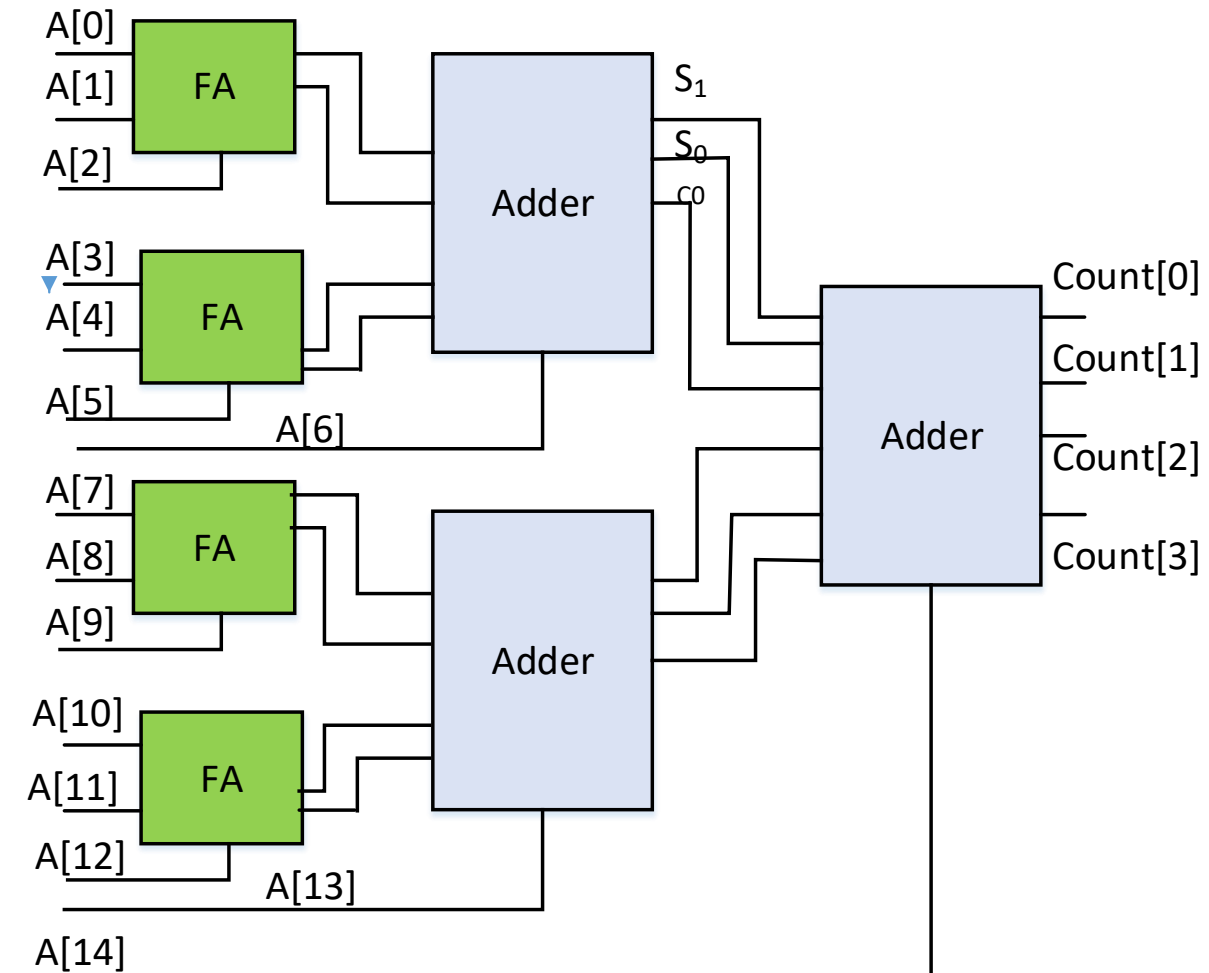
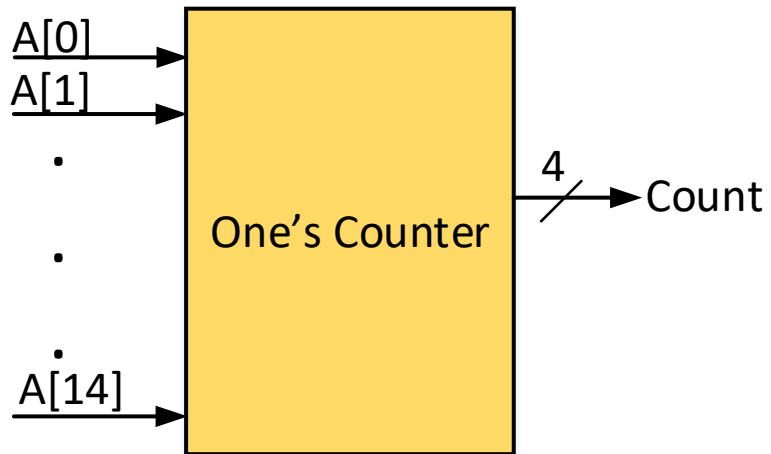


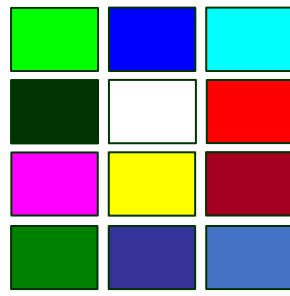


Special Combinational Circuit

Iterative Circuits

One's Counter



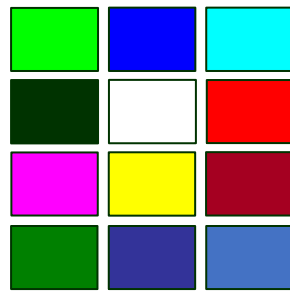


Special Combinational Circuit

Iterative Circuits

One's Counter

```
module ones_counter(  
    input  logic [14:0] A,  
    output logic [3:0]  C  
);  
  
    always_comb begin  
        C = 4'b0000;  
        for (int i = 0; i <= 14; i++) begin  
            if (A[i] == 1'b1)  
                C = C + 1;  
        end  
    end  
endmodule
```



Conclusion

We have covered these topics:

- **Combinational components:**

- Decoder
- Multiplexer
- Comparator
- Adder
- Shifter
- ALU
- Case Study
 - Max Finder
 - One's Counter

- Encoder
- DeMux
- CLA, CSA, ...
- Rotator
- Multiplier, ...
- ...

